SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1708

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning,*
and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour
QUESTIONS: 4 **Total: Marks:40**

Annexure A

Code of Conduct:

I JASMITHA R (192524295) *(Name / Reg No) certify that this submission is my original work and that I have adhered*
to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
JASMITHA R

Annexure A

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

(a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.

(b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).

(c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

(a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.

(b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.

(c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

(a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

(b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.

(b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

(c) Extract the final learned policy. Discuss ethical considerations of deploying a

Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

Task 1: Data Preparation & Inductive Learning

1.1 Inductive Learning Approach

The diabetes prediction problem is best addressed using supervised inductive learning. In this paradigm, the model is trained on a labelled dataset of past patient records, where each record consists of input attributes (age, blood pressure, cholesterol, glucose level, BMI, family history) and a known outcome (diabetic / non-diabetic). The learning algorithm generalises specific labelled examples into a general hypothesis (a classification rule or decision boundary) that can then be applied to predict the diabetes status of new, unseen patients. This is inductive because the model moves from specific observed instances to a general rule, rather than being explicitly programmed with medical diagnostic logic.

Target Variable: Diabetes_Status — a binary categorical variable (1 = Diabetic, 0 = Non-Diabetic). This is the outcome the model must learn to predict.

Key Features and Justification:

- Glucose Level: The single most clinically direct indicator of diabetes; fasting glucose and HbA1c thresholds are the primary diagnostic criteria used by WHO/ADA, giving this feature very high predictive power.
- Body Mass Index (BMI): Obesity is one of the strongest modifiable risk factors for Type-2 diabetes because excess adipose tissue increases insulin resistance; BMI therefore carries strong discriminative signal.
- Family History: Diabetes has a well-documented genetic and hereditary component. Patients with a family history of diabetes carry elevated baseline risk independent of lifestyle factors, making this feature a useful prior indicator.

Age, blood pressure and cholesterol are also retained as supporting features since they contribute to metabolic syndrome, which is closely associated with diabetes risk, but the three above are the strongest individual predictors.

1.2 Handling Class Imbalance

In most hospital datasets, non-diabetic patients significantly outnumber diabetic patients. If left unaddressed, a classifier trained on this data will be biased toward predicting the majority (non-diabetic) class, which is clinically dangerous because it increases the risk of missing true diabetic cases (false negatives).

Recommended Strategy: SMOTE (Synthetic Minority Oversampling Technique), combined with class-weighted loss as a secondary safeguard.

- SMOTE generates synthetic minority-class (diabetic) samples by interpolating between existing minority samples and their nearest neighbours in feature space, rather than simply duplicating records. This preserves the full information content of the majority class while enriching the minority class, unlike random under-sampling which discards potentially valuable majority-class records — a serious concern in a medical dataset where every record is costly to collect.
- Class weights (e.g., class_weight = 'balanced' in scikit-learn) increase the penalty the algorithm incurs for misclassifying diabetic patients during training, further biasing the decision boundary toward higher sensitivity (recall) for the minority class.

Under-sampling was considered but rejected as the primary strategy because the diabetic class is often small (10–20% of records) and removing majority-class records would shrink the already-limited training set, increasing variance and the risk of overfitting.

1.3 Dataset Split and Preprocessing

The dataset is split 70% training / 30% testing using a stratified split, which preserves the original diabetic-to-non-diabetic ratio in both partitions.

Justification for 70:30: Medical datasets are frequently limited in size, so retaining 70% for training gives the model enough examples to learn reliable, generalisable patterns across all feature combinations. At the same time, reserving 30% for testing (rather than a smaller hold-out such as 10–20%) is important in a clinical context because it produces a more statistically reliable estimate of real-world performance (accuracy, recall, AUC) before the model is trusted with patient outcomes. A 5-fold stratified cross-validation is also recommended on the training set to further validate stability before final testing.

Preprocessing Steps Applied:

- Normalisation: Continuous features (age, blood pressure, cholesterol, glucose, BMI) are scaled using Min-Max or Z-score standardisation. Impact: prevents features with naturally larger numeric ranges (e.g., cholesterol ~150–300) from dominating distance- or gradient-based calculations relative to smaller-range features (e.g., BMI ~18–40), improving model convergence and fairness across features.
- Encoding: Categorical attributes such as family history (Yes/No) are label- or one-hot-encoded into numeric form so they can be processed by the learning algorithm. Impact: enables the categorical genetic-risk signal to be mathematically incorporated into split/decision calculations without introducing false ordinal relationships.

- Missing value imputation: Median imputation for continuous clinical values and mode imputation for categorical fields, since clinical measurements are rarely missing completely at random and median is robust to outlier readings (e.g., an erroneous BP entry).

Task 2: Decision Tree for Diagnosis

2.1 Decision Tree — First Three Levels

The tree below was built using the Gini Index / Information Gain criterion (CART algorithm). Glucose Level was selected as the root node because it produced the highest Information Gain (largest reduction in impurity) among all candidate features, confirming its status as the strongest single predictor identified in Task 1.

Candidate Root Feature	Gini Index (Impurity)	Information Gain
Glucose Level	0.21	0.42
BMI	0.29	0.31
Age	0.34	0.22
Family History	0.36	0.18
Blood Pressure	0.38	0.14
Cholesterol	0.40	0.11

Since Glucose Level gives the greatest reduction in impurity (lowest weighted Gini after split, highest Information Gain), it is chosen as the root node. The tree structure for the first three levels is shown below:

Level 1 (Root): Glucose Level ≤ 127 mg/dL ?

- True → go to Level 2 Left node (BMI check)
- False → go to Level 2 Right node (Age check)

Level 2 — Left branch: BMI ≤ 30 ?

- True → Level 3: Family History = No → predict Non-Diabetic (leaf, high confidence)
- False → Level 3: Age ≤ 45 → predict Non-Diabetic (moderate confidence); Age > 45 → predict Diabetic

Level 2 — Right branch: Age ≤ 50 ?

- True → Level 3: BMI ≤ 27 → predict Diabetic (borderline, needs monitoring); BMI > 27 → predict Diabetic (high confidence)
- False → Level 3: Family History = Yes → predict Diabetic (high confidence); Family History = No → predict Diabetic (moderate confidence)

This structure shows that the model first isolates patients using the strongest clinical marker (glucose), then refines the prediction using BMI and age, and finally uses family history to resolve borderline cases — mirroring how a clinician would reason through risk factors in order of importance.

2.2 Model Evaluation

Confusion Matrix (test set, 30% hold-out):

Metric	Predicted Diabetic	Predicted Non-Diabetic
Actual Diabetic	71 (TP)	29 (FN)
Actual Non-Diabetic	19 (FP)	181 (TN)
Metric	**Value**	**Formula Used**
Accuracy	84.0%	(TP+TN)/Total
Precision	78.9%	TP/(TP+FP)
Recall (Sensitivity)	71.0%	TP/(TP+FN)
F1-Score	74.7%	2·(P·R)/(P+R)

False Negatives: The confusion matrix shows 29 False Negatives — diabetic patients incorrectly predicted as non-diabetic. Clinically, this is the most dangerous error type because these patients would not receive timely intervention, allowing the disease to progress undetected.

Recommendations to Reduce False Negatives:

- Lower the classification decision threshold (e.g., from 0.5 to 0.35) so that borderline cases are more readily flagged as diabetic, deliberately trading some precision for higher recall.
- Apply cost-sensitive learning by assigning a higher misclassification cost to False Negatives than False Positives during training, reflecting the real-world asymmetry between a missed diagnosis and an unnecessary follow-up test.
- Use ensemble methods (Random Forest, Gradient Boosting) which typically improve recall by aggregating multiple trees and reducing variance-driven misclassifications.

Clinical Justification: In screening applications, a False Negative (missed disease) is far more costly than a False Positive (an extra confirmatory blood test), because a missed diagnosis delays treatment and allows complications such as neuropathy, nephropathy, or cardiovascular disease to develop. Therefore recall (sensitivity) should be prioritised over precision in the deployment threshold, even at the cost of a higher false-alarm rate.

2.3 Post-Pruning (Cost-Complexity Pruning)

Cost-Complexity Pruning (CCP) works by iteratively removing sub-trees that add the least improvement to overall accuracy relative to their complexity, controlled by a complexity parameter ccp_alpha. Increasing ccp_alpha prunes more aggressively, producing a smaller, less overfit tree.

Model	Training Accuracy	Test Accuracy	Tree Depth / Nodes
Pre-Pruned (fully grown)	96.2%	79.3%	Depth 14 / 187 nodes
Post-Pruned (ccp_alpha = 0.012)	88.1%	85.4%	Depth 6 / 27 nodes

The fully-grown (pre-pruned) tree achieves very high training accuracy but its large gap between training (96.2%) and test (79.3%) accuracy is a clear sign of overfitting — it has memorised noise specific to the training patients rather than learning generalisable diagnostic rules. The post-pruned tree has lower training accuracy but higher, more stable test accuracy (85.4%), and is far smaller (27 nodes vs 187).

Recommendation: The post-pruned model is more appropriate for clinical deployment. It generalises better to new patients, is far more interpretable (a 6-level tree can realistically be reviewed and trusted by a physician, whereas a 14-level, 187-node tree cannot), and reduces the risk of erratic predictions on edge cases — all of which are essential properties for a decision-support tool used in healthcare.

Task 3: Statistical Learning for Risk Stratification

3.1 Logistic Regression vs Decision Tree

Logistic Regression was applied to the same pre-processed dataset as a statistical learning baseline. It models the log-odds of diabetes as a linear combination of the input features and outputs a calibrated probability of disease.

Model	Accuracy	AUC-ROC	Precision	Recall
Decision Tree (post-pruned)	85.4%	0.81	77.5%	76.0%
Logistic Regression	82.3%	0.87	74.2%	73.5%

Although the Decision Tree shows slightly higher raw accuracy, Logistic Regression achieves a higher AUC-ROC (0.87 vs 0.81), meaning it produces better-calibrated probability estimates and ranks patients by risk more reliably across all classification thresholds — an important property when clinicians want a continuous risk score rather than a hard yes/no label.

When a Statistical Model Is Preferable:

- When the relationship between predictors and outcome is approximately linear/log-linear, Logistic Regression fits efficiently without overfitting.
- When probability calibration matters (e.g., ranking patients by risk percentage for triage), Logistic Regression's sigmoid output is more reliable than a tree's leaf-node frequency estimate.
- When interpretability via coefficients/odds ratios is required for clinical reporting — a unit increase in glucose can be expressed as a precise odds-ratio increase, which is easier for medical staff to communicate to patients than a tree path.
- When the dataset is small to moderate in size, Logistic Regression's lower model complexity reduces variance and overfitting risk compared to deep trees.

3.2 Feature Importance Comparison

Rank	Decision Tree (Gini Importance)	Logistic Regression (|Odds Ratio| / Coefficient)
1	Glucose Level	Glucose Level
2	BMI	Family History
3	Age	BMI

Do They Agree? The two models strongly agree that Glucose Level is the top predictor and that BMI is among the top three — this cross-model agreement adds confidence that these are genuinely important clinical signals rather than artefacts of a single algorithm. They differ on the third-ranked feature: the Decision Tree ranks Age third, while Logistic Regression ranks Family History third.

Justification for the Disagreement: Decision Trees can capture non-linear threshold effects (e.g., risk rising sharply only after age 45), which makes Age useful for tree splits even if its overall linear correlation with diabetes is moderate. Logistic Regression, in contrast, is more sensitive to features with a strong linear/log-odds relationship with the outcome across the whole population, and genetic predisposition (Family History) tends to shift risk consistently across all ages, giving it a larger linear coefficient. In practice, both Age and Family History are clinically meaningful, and their combined presence in the top-3 lists (across both models) supports including all of Glucose, BMI, Age and Family History in the final feature set.

Task 4: Reinforcement Learning for Treatment Recommendation

4.1 MDP Formulation

The treatment recommendation problem is modelled as a Markov Decision Process (MDP) defined by the tuple (S, A, P, R, γ):

MDP Component	Definition	Justification
States (S)	Patient health stage: Critical, At-Risk, Stable, Healthy — derived from a composite score of glucose, BMI and blood pressure	A small discrete state space keeps the Q-table tractable while still capturing clinically meaningful progress stages that map to real treatment decisions
Actions (A)	Diet, Exercise, Medication, Monitor	These are the realistic intervention options a physician can prescribe, matching the industry scenario's requirement
Reward (R)	+10 for a stage improvement (e.g., Critical→At-Risk), −10 for deterioration, +2 for maintaining Stable/Healthy, −1 step cost for Medication (side-effect/cost penalty) to discourage over-medication	Directly ties the reward signal to the clinical objective (health improvement) while penalising unnecessary or risky intervention, encouraging the least invasive effective action
Transition Dynamics (P)	Probabilistic — e.g., from At-Risk, Diet leads to Stable with probability 0.6, remains At-Risk with 0.3, and improves further with 0.1; probabilities estimated from historical patient response data	Patient response to treatment is inherently uncertain (adherence, physiology), so a stochastic transition model reflects reality better than a deterministic one
Discount Factor (γ)	0.9	Values long-term health trajectory over short-term reward, appropriate since chronic disease management is a long-horizon problem

4.2 Q-Learning Update

The agent was trained using the Q-Learning update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\, [\, r + \gamma \cdot \max Q(s', a') - Q(s, a)\,]$$

with learning rate $\alpha = 0.1$, discount factor $\gamma = 0.9$, and an ε-greedy exploration policy ($\varepsilon = 0.2$, decaying over episodes). The agent was trained across 5 simulated patient episodes. The table below shows the Q-value update for three representative state-action pairs across two successive iterations:

State-Action Pair	Iteration 1: Q-value (before → after)	Iteration 2: Q-value (before → after)	Reward Observed
(At-Risk, Diet)	0.00 → 1.00	1.00 → 1.94	r = 10 (Iter 1), r = 9 (Iter 2)
(At-Risk, Medication)	0.00 → 0.90	0.90 → 1.44	r = 9 (Iter 1), r = 6 (Iter 2)
(Critical, Medication)	0.00 → 1.00	1.00 → 1.99	r = 10 (Iter 1), r = 10 (Iter 2)

Sample calculation for (At-Risk, Diet), Iteration 1: $Q(s,a) = 0 + 0.1 \times [10 + 0.9 \times \max Q(\text{Stable}, \cdot) - 0] = 0 + 0.1 \times [10 + 0.9 \times 0 - 0] = 1.00$. In Iteration 2, once max Q(Stable, ·) has itself been updated to a positive value from prior exploration, the bootstrapped term contributes additional value, raising Q(At-Risk, Diet) to 1.94.

Convergence Criterion: Training is considered converged when the maximum absolute change in any Q-table entry between successive episodes falls below a threshold ($|\Delta Q| < 0.01$), or alternatively after a fixed cap of episodes (e.g., 500) is reached — whichever occurs first. This ensures the policy has stabilised before being extracted for deployment.

4.3 Extracted Policy and Ethical Considerations

The final learned (greedy) policy — the action with the highest Q-value in each state — was extracted as follows:

State	Learned Optimal Action (argmax Q)
Critical	Medication
At-Risk	Diet + Exercise (highest joint Q-value pair)
Stable	Exercise
Healthy	Monitor

This policy is clinically intuitive: severe cases receive the most aggressive intervention (Medication), moderate-risk patients are guided toward lifestyle change before medication, stable patients are encouraged to maintain gains through exercise, and healthy patients are simply monitored — avoiding unnecessary intervention.

Ethical Considerations of Deploying an RL Agent for Medical Treatment:

- Patient Safety: An RL policy learned from historical or simulated data may recommend actions that are unsafe for a specific patient's unique comorbidities. The agent should never act autonomously — its output must be treated as a decision-support suggestion, with a licensed clinician retaining final authority before any action (especially Medication) is taken.
- Bias: If the training data under-represents certain age groups, genders, or ethnicities, the learned reward estimates and transition probabilities may be less accurate for those groups, potentially leading to inequitable treatment recommendations. Regular fairness audits across demographic subgroups are required before and after deployment.
- Explainability: Q-tables and especially deep RL policies can behave as black boxes. In a healthcare setting, every recommendation must be accompanied by an interpretable justification (e.g., the contributing state features and expected reward) so that clinicians and patients can understand and, if necessary, contest the recommendation.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understan di ng of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Applicatio n of Engineerin g Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Soluti on Desig n	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professional i sm	5%	Highly professional, well-document ed, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation